

Federation-Based Deep Reinforcement Learning Cooperative Cache in Vehicular Edge Networks

Honghai Wu^{1b}, Jichong Jin^{1b}, Huahong Ma^{1b}, and Ling Xing

Abstract—With the emergence of a large number of computing resource-intensive applications and a variety of content delivery services, data in Internet of Vehicles (IoV) is exploding. In order to improve the service performance of IoV, vehicle edge computing (VEC) accelerates the response process of content requests and reduces the backhaul burden of the base station by caching content at the nodes of the edge network. However, the existing caching strategies are usually affected by high-computing and communication overhead, and cannot well capture the dynamic changes and content popularity of the vehicle network. In order to solve these problems, we design a novel Cooperative Caching scheme by using Mobility Prediction and Consistent Hash for federated learning (called CMCF), which integrates mobility prediction and consistent hashing into the content caching scheme, and uses the federated learning framework to optimize the cached content, then we use the deep reinforcement learning algorithm to develop the optimal cooperative caching policy to reduce the average delay of content transmission. Extensive simulation results prove the superiority of our method. Compared with other advanced caching schemes, CMCF can increase the cache hit rate by 8.7% and reduce the average content delivery delay by 17.8%.

Index Terms—Consistent hashing, cooperative caching, deep reinforcement learning, federated learning.

I. INTRODUCTION

THE VEHICLE edge network composed of a large number of mobile vehicles and roadside units (RSUs) is an important part of intelligent transportation system (ITS). In addition to improving the travel condition of the vehicle, the in-vehicle edge network can also provide information and entertainment services to the users in the vehicle [1]. These services often have high requirements for computing speed, communication quality and storage resources. Moreover, in a highly dynamic

environment, stable vehicle-to-vehicle (V2V) and vehicle-to-infrastructure (V2I) connections cannot be guaranteed [2]. At the same time, the high mobility of vehicles and limited communication distance lead to frequent interruption and reconstruction of connections, which will increase network overhead and retrieval delay [3]. Considering the timeliness and reusability of content in the network, caching in advance on the network edge carrier and migrating part of the cloud cache to the edge device can effectively reduce the transmission and calculation of repetitive content and enable users to obtain higher Quality of Experience (QoE) [4]. However, because the cache capacity of edge nodes is limited, how to design a cooperative cache scheme to make effective use of the cache space of all edge nodes has aroused great interest in academic circles.

Caching strategies in vehicle edge networks can be divided into noncooperative caching and collaborative caching [5]. In the noncooperative cache, node independently determines its own cache content and cache policy, and will not advertise the cache content information to other nodes, which will lead to frequent cache updates, cache redundancy and other problems. On the contrary, collaborative caching avoids these problems, because collaborative caching allows content sharing among multiple nodes and coordinates cache resources, which makes up for the shortcomings of noncollaborative caching. For example, some studies have considered the spatial cooperative caching in a double-layer heterogeneous network composed of edge servers (ESs) and cache nodes, and established a spatial collaborative caching strategy based on random geometry theory [6]. In order to achieve efficient resource aggregation of ES, Xu et al. [7] combined collaborative caching with DRL to build an intelligent edge caching framework to achieve collaborative caching between cloud and ESs. In addition, some studies consider the environment with limited edge resources, carry out collaborative caching on vehicles and RSU [8], [9], [10], which effectively optimizes the cache space utilization of edge nodes, reduces content acquisition time and improves user experience. But there are also some problems with these collaboration strategies. First, due to the differences in computing resources and cache capabilities of different nodes, joint resource scheduling becomes a complex task. Second, some video content can be compressed by specific processing, but it requires some computing resources. Then, how to efficiently allocate cache resources among vehicle edge nodes has not been well solved. Therefore, there are still many obstacles to the formulation of collaborative caching strategy.

Manuscript received 14 June 2023; accepted 30 June 2023. Date of publication 5 July 2023; date of current version 8 January 2024. This work was supported in part by the National Natural Science Foundation of China under Grant 62272146, Grant 62071170, Grant 62171180, and Grant 62072158; in part by the Program for Innovative Research Team in University of Henan Province under Grant 21IRTSTHN015; in part by the Key Science and the Research Program in University of Henan Province under Grant 21A510001; in part by the Henan Province Science Fund for Distinguished Young Scholars under Grant 222300420006; in part by the Science and Technology Research Project of Henan Province under Grant 222102210001; and in part by the Leading Talent in Scientific and Technological Innovation in Zhongyuan under Grant 234200510018. (Corresponding author: Honghai Wu.)

The authors are with the School of Information Engineering, Henan University of Science and Technology, Luoyang 471000, China (e-mail: honghai2018@haust.edu.cn; 200320050364@stu.haust.edu.cn; mhh@haust.edu.cn; xinglingmy@haust.edu.cn).

Digital Object Identifier 10.1109/JIOT.2023.3292374

In addition, due to the constant change of the position of the vehicle, the design of cooperative caching strategy in vehicle networks is also faced with the following four problems.

- 1) The popularity and effectiveness of regional content is time-varying. Caching strategies need to pay full attention to dynamic changes in cache space and content popularity, and dynamically determine cache content and cache location.
- 2) Tens of thousands of vehicle data will bring a lot of computing, communication and “Curse of Dimensionality” problems.
- 3) The restrictions of privacy protection rules. Due to the principle of privacy protection, user data cannot be transmitted and used at will. As a result, the centralized algorithm cannot get the global optimal caching policy of the vehicle network.
- 4) Cooperation between nodes. In the dynamic vehicle network, the appropriate cache nodes can be selected through the moving trajectory of the vehicle, but the problem of how to distribute the cache content among these nodes has not been well solved [11], [12].

In order to solve the above challenges, the coordination and storage space between nodes must be effectively utilized. In this article, we propose a new cooperative caching strategy CMCF in the vehicle edge network, which uses mobility prediction, federated learning and deep reinforcement learning when making caching decisions. Specifically, we use prediction based on partial matching (PPM) to predict the future path of each vehicle, and clustering vehicles with similar future paths into a cluster. Then, we use the federated learning framework to learn the regional content popularity model, which reduces data transmission through user local data training and better protects users’ data security and privacy. Finally, the optimal cooperative caching strategy is determined by double depth Q network (DDQN) to minimize the average delay of content transmission. (In this article, the words node and vehicle can be used interchangeably, because the vehicle is actually the node in the vehicle environment.)

The main contributions of this article are summarized as follows.

- 1) We use consistent hashing to allocate different cache contents in the established dynamic vehicle cluster to improve the utilization of cache resources.
- 2) A popular content prediction algorithm based on the combination of auto encoder (AE) and federated learning is proposed, in which AE can well obtain dynamic content popularity, and FL can reduce communication overhead by distributed training model.
- 3) A cooperative caching framework is designed through DRL. Cache nodes determine the optimal cooperative caching strategy through DDQN to minimize the average delay of content transmission.
- 4) We verify the effectiveness of our algorithm through a large number of simulations and comparative analysis.

The remainder of this article is arranged as follows. In Section II, we discuss the related work. Section III briefly introduces the system model. In Section IV, we introduce the design of CMCF scheme in detail, and then propose

an optimal cooperative caching strategy based on deep reinforcement learning. We test our proposed CMCF strategy in Section V, and then summarize the work of this article in Section VI.

II. RELATED WORK

In this section, we introduce the related work, which mainly includes collaborative caching and reinforcement learning-based caching.

Traditional Collaborative Caching: Many researchers have devoted themselves to designing different optimal caching strategies. Considering the social attributes, mobility and traffic patterns of vehicles, a cooperative caching scheme based on social attributes and vehicle trajectory prediction is proposed in [11]. Game theory is used in [13] to determine the best collaborator and the size of the cached data, in which interconnected vehicles equipped with storage resources can act as cache entities to cache hot content locally, which brings the advantage of using their own resources. Wang et al. [14] used LSTM model to predict the number of content requests on different processing modes and input time series, and proposes a cooperative caching strategy for Internet of Vehicles (IoV) content request prediction based on reinforcement learning to make it suitable for IoV scenarios. Zhang et al. designed a cooperative caching scheme in [15], which realizes the cooperation between different ESs in the case of limited cache resources, so as to effectively reduce the content delivery delay. In order to reduce the delay of content acquisition in the Internet of Things, Huang et al. [16] proposed a delay-aware content caching (DCC) algorithm, which jointly optimizes the cooperation between vehicles and RSU. Zhao et al. [17] proposed a joint optimization of edge caching and computing management for service caching, request scheduling, and resource allocation strategies to minimize the time-average service response latency of randomly arrived service requests. Due to the resource-constrained network environment (such as limited cache capacity and node battery) and the secure transmission of cache data, Gupta et al. [18] proposed a cooperative caching scheme in a hierarchical network structure. Yao et al. [19] proposed a cooperative caching scheme for vehicle networks, in which caching decisions are based on predicted node mobility. According to the past trajectory of the mobile node, they use partial matching to predict the probability of the mobile node reaching the hot spot area. This method helps the network select nodes that stay in hot spots for a long time to cache hot content. Although the above studies have made contributions, they cannot achieve the balance of cache availability, and any movement of cache nodes may disturb the cache structure of the whole system. In this article, the consistent hashing algorithm and DRL are used to ensure the balance between cache content and cache availability, and reduce the traffic overhead of mobile nodes.

Collaborative Caching Based on Learning Algorithm: With the continuous research and development of deep learning of reinforcement learning, a large number of scholars use learning algorithms to find the optimal cache decision. Qiao et al. [20]

proposed a new scheme for joint optimization of macro base station RSU and intelligent vehicles, and proposed a cooperative cache scheme based on DDPG. In order to realize collaborative caching, a vehicle social relationship model is constructed by using digital twin technology, and the potential social relations of vehicles in user preference similarity and content delivery are used to improve vehicle service quality [21], the optimal cache scheme of deep learning enabling is also proposed, but it still fails to solve the negative impact of centralized learning. A cooperative edge caching strategy based on federated deep reinforcement learning is proposed in [22] and [23], which enables Internet of Things devices to collaborate to learn and share the model. However, this algorithm is not suitable for highly dynamic vehicle edge networks. Ndikumana et al. [24] proposed infotainment caching in self-driving vehicles, in which a deep learning model for user feature prediction is considered to make caching decisions. Then, the optimization problem is used to improve the delay of Web content retrieval. Although the scheme has high accuracy in predicting the cache content, the data security is not considered in the process of data transmission. The method based on matrix factorization (MF) can achieve better results in terms of hit rate, delay reduction and high satisfaction with users. The work in [25] uses NMF to predict user preferences, and then proposes a mixed integer linear programming algorithm to optimize the cache strategy through prediction. Yu et al. [26] used federated learning to deploy a context-aware antagonistic automatic encoder (C-AAE) model on self-driving vehicles to predict content popularity, and improve the cache hit rate by precaching the requested content to the RSU that the vehicle arrives at the next time by considering the speed of the vehicle and the service range of the RSU. Ma et al. [27] based on the deep reinforcement learning algorithm, the best methods of vehicle content precaching and vehicle task allocation are proposed. Although learning algorithms are used in the above work, however, the learning-based caching scheme also has some problems, such as the centralized learning algorithm, has certain requirements on the computing power of the central node and brings high-communication overhead. And the centralized learning algorithm will also bring a series of problems, such as user data security and user privacy. The federation-based reinforcement learning strategy proposed in this article well deals with the problems not fully solved in the above article, and trains the data in a distributed way through the federation framework, which reduces the extra overhead caused by data transmission. It improves the security of user data and effectively protects user privacy data.

III. SYSTEM MODEL

In this part, we introduce the topology of the vehicle edge network that supports collaborative caching at first, and then discuss the communication model.

A. Network Model

We consider an urban traffic network model, which consists of a set of RSUs $\mathcal{N} = \{0, 1, 2, \dots, N\}$, and a number

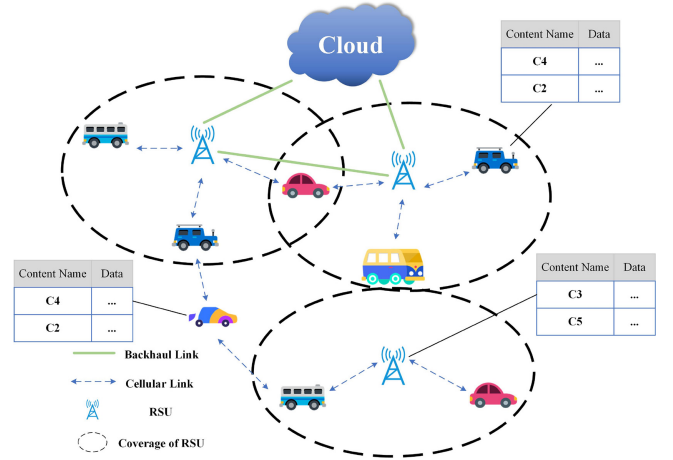


Fig. 1. Vehicle network model.

TABLE I
SYSTEM PARAMETERS

Notation	Meaning
Y	The number of cached content
$\mathcal{C}$	The name of cached content
V_k	The k -th vehicle
P_n	The n -th grid in location
P	Trip sequence of a vehicle
$\mathbb{N}_k$	The Bridging Centrality of the k -th vehicle

of vehicles $\mathcal{V} = \{0, 1, 2, \dots, V\}$ and a cloud server, where V_k represents the k th vehicle., as shown in Fig. 1 communication between vehicles is established through 802.11p dedicated short-range communication technology. Each vehicle can communicate wirelessly with neighboring vehicles in the cluster, or communicate wirelessly with RSUs within its communication range. RSUs are connected through optical fiber communication, and ESs for computing and caching are deployed, so the RSUs can cache various content to meet the content service needs of vehicle users. When a vehicle needs content that is not cached by itself, the vehicle first sends a request containing the name of the desired content and broadcasts it to neighbors. The vehicle will look for a nearby vehicle or RSU that can meet the request service, and then establish a communication to obtain the content, otherwise the request content can only be obtained from the nearby RSU or directly from the cloud server.

Assume that each vehicle maintains its cached content and cluster-related data, that is, caching popular content and recording neighbor information within the cluster, where the content index set is represented as $Y = \{0, 1, 2, \dots, y\}$, the content name is represented as $\mathcal{C} = \{C_1, C_2, \dots, C_S\}$. Each RSU maintains a list of content caches for vehicles within its scope of service. When a data block is received, each node decides whether to cache it according to its own replacement strategy. Therefore, the request content of the vehicle can be provided by a plurality of nodes. Some key parameters in the system model are listed in Table I.

B. Communication Model

In the cooperative caching strategy proposed in this article, the speed of obtaining content from different locations

is different, because the vehicles in the cluster communicate through wireless links, and the communication between RSUs uses wired links. By default, we ask users to choose the method with the fastest transfer rate to obtain content. The requesting vehicle can fetch content in three ways so the channel gain of the k th vehicle V_k is modeled as [28]

$$h_k(\tau(x, V_k)) = \partial_k(\tau(x, V_k))\xi_k(\tau(x, V_k)) \quad (1)$$

where $x = \{V, S, C\}$ contradistinction represents the vehicles, RSU, and cloud within the cluster, $\tau(x, V_k)$ is the distance between the vehicle, RSU, and cloud server and V_k in the cluster, $\partial_k(\tau(x, V_k))$ is the path loss between the vehicle, RSU, and cloud server and V_k in the cluster, $\xi_k(\tau(x, V_k))$ is the shadowing channel fading between the vehicle, RSU, and cloud server and V_k in the cluster.

When V_k makes a content request, V_k first checks to see if the requested content is cached locally, and if so gets the content directly. Otherwise, you need to check if the collaborative vehicles in the cluster have cached the request content.

Then, according to Shannon's theorem, the transmission rate between the cooperative vehicle, and V_k in the cluster is calculated as follows [29]:

$$R_{V,k} = B \log_2 \left(1 + \frac{P_V h_k(\tau(x, V_k))}{\sigma^2} \right) \quad (2)$$

where B is the transmission bandwidth and σ^2 is Gaussian white noise. P_V is the transmission power of vehicle V2V communication in the cluster.

Similarly, if the vehicle in the cluster does not cache the request content, the vehicle sends a content request to the RSU within its range. If the RSU cache contains the request content, the transfer rate between the local RSU and V_k is

$$R_{R,k} = B \log_2 \left(1 + \frac{P_B h_k(\tau(x, V_k))}{\sigma^2} \right) \quad (3)$$

where P_B is the transmit power of the local RSU.

Similarly, the transfer rate between the cloud server and vehicle V_k is

$$R_{C,k} = B \log_2 \left(1 + \frac{P_C h_k(\tau(x, V_k))}{\sigma^2} \right) \quad (4)$$

where P_C is the transmit power of the cloud server.

IV. DRL COOPERATIVE CACHING SCHEME BASED ON DYNAMIC FEDERATION

The CMCF scheme consists of the following modules as shown in Fig. 2: dynamic cluster construction, content distribution, and popular content acquisition, and finally uses the DRL algorithm to make the optimal collaborative caching strategy. In the module of dynamic cluster construction, RSU allocates vehicles with similar movement directions and future trajectories in the same area to a cluster, and selects appropriate cluster head nodes according to the centrality of the bridge. In the module of content allocation, the cached content is allocated in each node through the hashing algorithm. In the module of popular content acquisition, an algorithm combining federated learning with automatic encoder is proposed to obtain popular content. In the formulation of DRL optimal policy, we use DDQN to obtain the optimal cache policy.

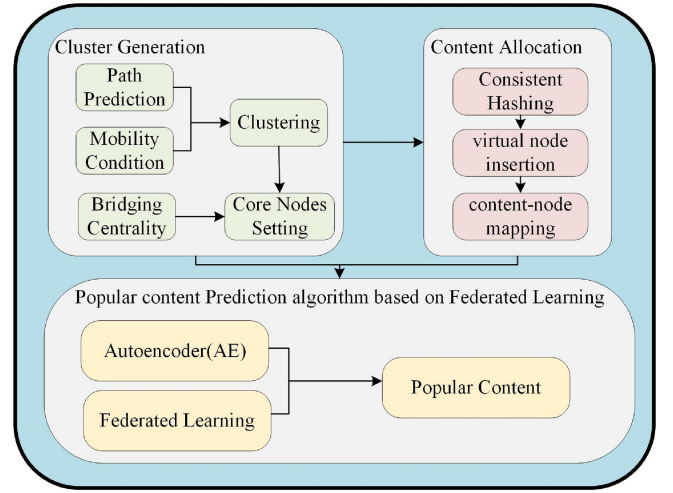


Fig. 2. Architecture of the CMCF scheme.

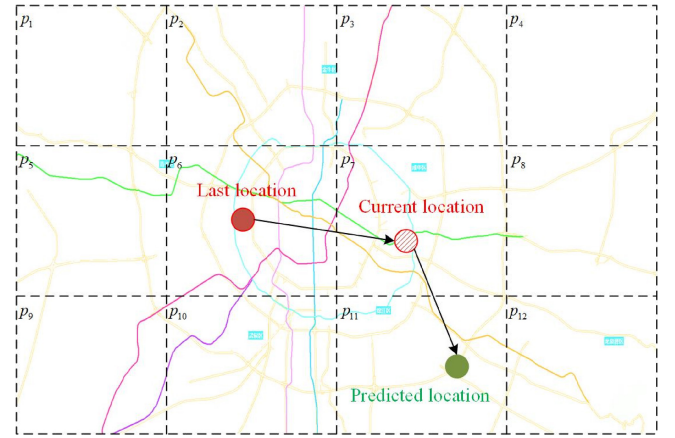


Fig. 3. Grid division and mobility prediction.

A. Dynamic Cluster Construction

In order to make the nodes work together efficiently, our proposed cluster construction algorithm can not only maintain the stability of the cooperative cache node link, but also deal with the dynamic changes caused by vehicles joining/leaving the cluster.

1) *Cluster Formation*: In this article, the dynamic vehicle cluster is divided by the key attributes of the vehicle, mainly including the moving direction and future trajectory of the vehicle. In order to accurately predict the approximate position of the vehicle at the next moment, we do not use a specific path [30], and divide the map into grids of uniform size, as shown in Fig. 3. A grid is used to represent the approximate position of the vehicle at a certain moment, and the grid contains multiple drivable roads. The moving path of the vehicle is represented by a set of grids, $P = \{P_1, P_2, \dots, P_n\}$, where P_n is n th grids. Here, we make a prediction of the grid where the vehicle may be at the next moment through PPM [31].

At first, we can establish a tree structure by collecting vehicle tracks. Except for the root node, each node in the tree contains two symbols, representing the number of times P_n and P_n appear in the sequence of the n th grid. Multiple elements P_n form a travel path. For existing paths, increment the counter

after the element appears. New paths are represented by adding new branches to the tree species. Three elements can represent a path sequence, for example $\{P_1, P_2, P_3\}$. Given path sequence $\{P_7, P_5, P_2, P_3, P_5, P_2, P_8, P_5, P_2, P_8\}$, According to the given path sequence, $\{P_7, P_5, P_2\}$, $\{P_3, P_5, P_2\}$, and $\{P_8, P_5, P_2\}$, are added to the tree. Similarly, the sequence starting with $\{P_5, P_2\}$ appears several times, increase the counter of the $\{P_5, P_2\}$ element as described above.

Then, given two consecutive grids P_{m-1} and P_m , if you can parse $\{P_{m-1}, P_m\}$ in the tree, then it can be predicted that the next slice vehicle is most likely to reach grid P_{m-1} . For example, given $\{P_{m-1}, P_m\} = \{P_5, P_2\}$, we can know from the path sequence that, the probability that P_3 and P_8 are P_{m+1} is $1/3$ and $2/3$, respectively. Usually the grid with a higher probability is the position we predict the vehicle will be in at the next moment, so the grid P_8 is the predicted vehicle position.

After the cluster is established, we need a stable cluster head node to maintain the cluster. We use bridging centrality as a measure of vehicle connectivity, because bridging centrality indicates connectivity with neighbor nodes, and nodes with high-bridging centrality have better connectivity and stability. We choose the vehicle V_k with the highest bridge center in the cluster as the cluster head node. In social network analysis, centrality measures are often used to identify the most influential nodes or key infrastructure nodes in a social network. Bridging centrality is used to determine the importance of vehicles in the network to form connections between nodes. For vehicle V_k , the bridging centrality is as follows:

$$N_k = h_k \cdot \frac{|\mathbb{R}_k|}{\mathbb{R}} \quad (5)$$

where $\mathbb{R}$ is the connected number of all nodes in the cluster, h_k is the number of nodes that can be connected with the vehicle k , which is called the degree of the vehicle k , And $\mathbb{R}_k$ is a set of paths through V_k among the paths included in $\mathbb{R}$ possible paths among all neighbors ($\mathbb{R}_k \subseteq \mathbb{R}$). The $(|\mathbb{R}_k|/\mathbb{R})$ term in (5) is the bridge centrality of each node.

If there are multiple nodes with the same bridging centrality in the service range of the current RSU, we choose the node with the smallest physical distance to act as the cluster head node. The vehicle will join the nearest cluster, while ensuring that the nodes in the cluster are within the communication range of the cluster head.

2) *Cluster Management*: Due to the mobility of vehicles, it often happens that vehicles leave the current RSU service area or new vehicles join the current RSU, so the vehicles in the cluster are not fixed. When the node in the cluster finds that it is unable to establish a connection with all the nodes in the original cluster, it shows that the node has left the original vehicle cluster and become an isolated node. At this point, the node will send a request to RSU to ask RSU to assign the node to a neighboring cluster similar to its own future trajectory. If the cluster head node leaves the current cluster, the corresponding RSU immediately selects a new cluster head node through the method described above. At certain moments, such as morning and evening vehicle peak hours, a large number of vehicles join the cluster. When the number

of nodes in the cluster doubles, RSU will divide the current cluster into two new clusters and select a new cluster head node.

B. Content Allocation Based on Consistent Hashing Algorithm

Due to the limited amount of popular content, the problem of duplicate cache content may occur on the adjacent nodes in the cluster. in order to reduce the occurrence of such problems, this article considers using the consistent hashing algorithm on the nodes in the cluster to reduce cache redundancy. Although the basic hash algorithm can map the relationship between service requests and service processors to achieve the purpose of dynamic allocation. However, the defects of this method are also obvious. First of all, it cannot load balance the entire system, which may cause some servers to be too busy to handle new requests, while other servers are too idle, resulting in overall resource utilization of the system. Second, when a server in a distributed cluster goes down, it will directly cause some service requests to be unprocessed. This situation is very bad in a distributed system, which will cause unbalanced load and high-data migration costs. A perfect distributed system should have good balance and monotonicity, that is, nodes are evenly distributed in the hash space, and the addition and removal of nodes will not cause a large number of hash relocation, and consistent hashing can just meet these conditions. Consistent hashing has predictable scalability, data replication and partitioning, more uniform load distribution, and better availability and fault tolerance. Therefore, consistent hashing can achieve better load balancing and higher fault tolerance. Therefore, this article uses consistent hashing to optimize content distribution between nodes [32].

The hash ring is a virtual ring mapped by a consistent hash algorithm. the value of the hash ring consists of 0 to $2^{32} - 1$ integers. In the consistent hash, each cooperative node is mapped to a unique integer, corresponding to the specific location of the cooperative node on the hash ring. If a node is added or removed, only the clockwise adjacent successor nodes of the node on the hash ring will be affected, and other data will not be affected. The content on the hash ring is assigned to the nearest cooperative node in a clockwise direction, and there is a mapping relationship between the cache content and the nodes on the hash ring. In addition, in order to achieve cache load balancing, it can be realized by adding virtual nodes on the hash ring [33].

As shown in Fig. 4, we describe the process of content allocation. There are five nodes on the hash ring shown in the figure, including three physical nodes x_1 , x_2 , and x_3 , and two virtual nodes x'_1 , x'_2 corresponding to x_1 and x_2 . Contents c_1 to c_6 are mapped at different positions of the hash ring. From the previous introduction, we know that c_1 is assigned to the nearest node x'_1 in a clockwise direction. Because x'_1 is a virtual node, c_1 is actually assigned to the real node c_1 corresponding to x'_1 . When x'_1 leaves the cluster, the content c_1 and c_5 that originally belonged to node x_1 will be assigned clockwise to the new node, the x_2 node. When the c_1 packet reaches its corresponding node in a clockwise direction, the node will

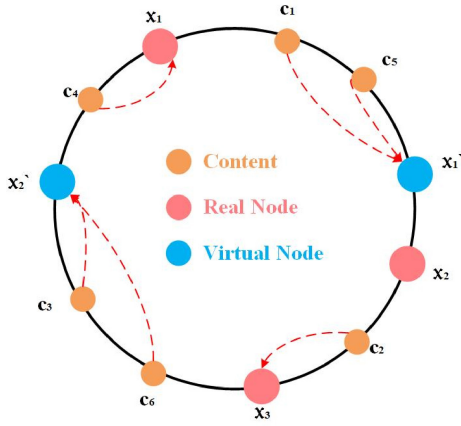


Fig. 4. Consistent hashing with virtual nodes.

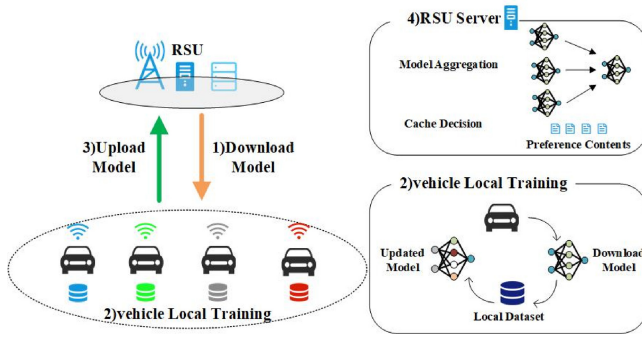


Fig. 5. Federal learning.

independently consider whether to cache the content and will not affect the cached content of other nodes.

Through the above steps, different content will be cached in the cache space near the collaboration node.

C. Federated Learning Based on Mobility

In this part, we propose a mobility-based FL framework to predict content popularity, FL keeps the training data on the vehicle, and facilitates the collaborative training of neural network models among vehicles under the coordination of the RSU server. It significantly reduces the privacy risk of vehicles and greatly reduces the communication cost brought by centralized ML [34]. FL is performed through multiple communication rounds (iterations).

As shown in Fig. 5, our proposed FL framework consists of the following five steps.

1) *Vehicle Selection*: The first task is to find the nodes in the cluster that can participate in FL and complete training (vehicles with sufficient computing and caching capabilities). In order to simplify the experimental process, we default that all cooperative nodes in the cluster can complete FL training.

2) *Model Download*: In this step, the cooperative nodes in the cluster participate in the FL training of r rounds of communication. The vehicles participating in the training download the global model from the RSU and use the local data to train the model. The detailed process is shown in Fig. 5.

3) *Model Training*: In this step, the participating training vehicles will use the local data to train the global model downloaded from the RSU and update the local model through training iterations, and the updated local model will be fed back to the local RSU. The AE model is trained in the above FL framework, which is used to predict the popularity of content and help us complete collaborative caching. The data training on the vehicle is completed through b iterations, and the data $\varphi_{k,b}$ participating in the training is obtained from the training set data of the vehicle V_k . The AE model includes encoders and decoders, where $\zeta_{k,b}^c$ and $\alpha_{k,b}^c$ represent the weight matrix and bias vector of the encoder b iterations, respectively, and $\zeta_{k,b}^d$ and $\alpha_{k,b}^d$ represent the weight matrix and bias vector of the decoder b iterations, respectively. So $\omega_{k,b} = \{\zeta_{k,b}^c, \alpha_{k,b}^c, \zeta_{k,b}^d, \alpha_{k,b}^d\}$ represents the local model of vehicle V_k at b iterations. To process the training data ϖ in $\varphi_{k,b}$, nonlinear logistic functions and activation logistic functions are used, respectively, [35]. In the encoder, the hidden features of the training data ϖ mapped to the hidden layer are obtained through the nonlinear activation function, as follows:

$$z(\varpi) = q(\zeta_{k,b}^c \varpi + \alpha_{k,b}^c). \quad (6)$$

The reconstructed input $\tilde{\omega}$ of the training data is calculated by the logical activation function in the decoder as follows:

$$\tilde{\omega} = p(\zeta_{k,b}^d \varpi + \alpha_{k,b}^d). \quad (7)$$

The loss function for calculating the training data ϖ through the local model $\omega_{k,b}$ is

$$l(\omega_{k,b}; \varpi) = (\varpi - \tilde{\omega})^2 \quad (8)$$

where $\omega_{k,1} = \omega$, then the local loss function of $\varphi_{k,b}$ is calculated by the following formula:

$$q(\omega_{k,b}) = \frac{1}{|\varphi_{k,b}|} \sum_{x \in \varphi_{k,b}} l(\omega_{k,b}; \varpi) \quad (9)$$

where $|\varphi_{k,b}|$ is the amount of data in $\varphi_{k,b}$.

Then, in order to reduce the deviation between the local model $\omega_{k,b}$ and the global model ω , this article solves the problem by calculating the regularized local loss function, the formula is as follows:

$$p(\omega_{k,b}) = q(\omega_{k,b}) + \frac{\rho}{2} \|\omega - \omega_{k,b}\|^2 \quad (10)$$

where ρ is the regularization parameter.

Let $\nabla p(\omega_{k,b})$ be the gradient of $p(\omega_{k,b})$, which is called local gradient. The local gradient of aggregation can be calculated as

$$\nabla \mathbb{Z}_{k,b} = \nabla p(\omega_{k,b}) + \psi \nabla p_k^d \quad (11)$$

where ψ is the attenuation coefficient, p_k^d is the local gradient.

Then the results of the next iteration of the local model are as follows:

$$\omega_{k,b+1} = \omega - \eta_l^r \nabla \mathbb{Z}_{k,b} \quad (12)$$

where η_l^r is the local learning rate of the r round, and the calculation formula is

$$\eta_l^r = \eta_l \max\{q, \log(r)\} \quad (13)$$

where η_l is the initial local learning rate. When one iteration is completed, the vehicle V_k will again sample some data from the training set for the next iteration. Until the iteration e times, we consider that the vehicle V_k model training is over, and then send the new model ω_k to the RSU where V_k is located.

4) *Upload Updated Model*: This step is to upload the local model of the vehicle to the RSU server, as shown in step (3) of Fig. 5. Compared with the computational cost, the communication cost is dominant in the FL process.

5) *Federated Aggregation*: After the vehicle uploads the model, the fifth step is to generate a new global model ω^r by calculating the weighted sum of all received local models $\omega_{k,b}$, and use the weighted average algorithm to calculate the model. The weight of parameter aggregation depends on the amount of local training data. RSU updates the global model ω^r by weighted average, as follows:

$$\omega^r = \omega^{r-1} + \frac{\vartheta_k}{\vartheta} \omega_k \quad (14)$$

where ϑ_k represents the amount of data in vehicle k , and ϑ is the sum of the data of all training vehicles. The latest global parameter model ω^r is used for the next round of training of vehicles in the cluster, collaborative vehicles predict popular content through an updated global model. There is no native data exchange between the vehicle and the RSU, which reduces the data transmission and protects the privacy.

D. Optimal Cooperative Caching Based on DRL

Through the above work, our goal in this section is to optimize our collaborative caching strategy through DRL to minimize the average delivery delay of content. Because the vehicle edge network is dynamic, the mobile cluster of vehicle nodes will also change dynamically, so the content cached in the edge nodes also needs to be dynamically updated and replaced. Since the cache space of edge nodes is very limited, in this section we use the DDQN algorithm to formulate an optimal cooperative cache strategy.

1) *DRL Framework*: We will formulate the DRL framework as follows, including state space, action space, and reward function.

a) *State space*: We regard the total cache content of the vehicle cluster and RSU as the current state s_t the content of state space s_t is arranged in descending order according to the popularity of the predicted popular content. Expressed as $s_t = \{s_{t1}, s_{t2}, \dots, s_{tm}\}$, where s_{tc} is the c th popular content at time t .

b) *Action space*: Action a_t indicates whether the total cache contents of the vehicle cluster and RSU need to be replaced. Among the predicted hot content, the content not cached in the total cache content constitutes a collection N . If $a_t = 1$, then randomly select n content from N to replace the content with the lowest popularity in the local cache, and then sort the content in descending order according to the popularity of the content to get s_{t+1} . If $a_t = 0$, the contents of the local cache will not be replaced.

c) *Reward function*: In order to improve the user's QOS, we need to maximize the system rewards, that is, minimize the delay in obtaining content. When the requested content is

cached in different locations, the content transmission delay of the requested content f is different (assuming that the content is the same size, the size is H).

If content f is stored locally in vehicle V_k , the transmission delay at this time is ignored.

If the content f is stored in the neighbor vehicle V_x in the cluster, the vehicle V_x transmits the content f to V_k . Therefore, the transmission delay is calculated as

$$J_{V,k}^f = \frac{H}{R_{V,k}}. \quad (15)$$

The transmission rate between vehicles in the cluster is obtained by (2).

If the content f requested by user V_k is cached in the RSU where the vehicle cluster is currently located, the user can directly establish a connection with the RSU, and obtain the requested content f through the current RSU. The delay in obtaining the content is expressed as

$$J_{R,k}^f = \frac{H}{R_{R,k}} \quad (16)$$

where $R_{R,k}$ can be calculated from (3) above.

If the content f is stored in adjacent RSU, the adjacent RSU will send the content to the local RSU, and the local RSU will forward the content to V_k . Therefore, the transmission delay is calculated as

$$\bar{J}_{R,k}^f = \frac{H}{R_{R,k}} + \frac{H}{R_{R-R}} \quad (17)$$

where, R_{R-R} represents the transmission rate of optical fiber communication between adjacent RSU.

If none of the neighboring edge nodes have cached the requested content, then the content f needs to be transferred from the cloud server to V_k , so the content transfer delay is as follows:

$$J_{C,k}^f = \frac{H}{R_{C,k}} \quad (18)$$

where $R_{C,k}$ can be calculated from (4) above.

As shown above, you can get the corresponding reward under different request acquisition methods. Therefore, the reward function for vehicle V_k to obtain the request content f at t time is defined as follows:

$$r_{k,t}^f = \begin{cases} e^{-\lambda_1 J_{V,k}^f}, & \text{V2V Communication} \\ e^{-\lambda_2 J_{R,k}^f}, & \text{Communication to R} \\ e^{-(\lambda_2 J_{R,k}^f + \lambda_3 \bar{J}_{R,k}^f)}, & \text{R - R Cooperation} \\ e^{-\lambda_4 J_{C,k}^f}, & \text{Cloud Service} \end{cases} \quad (19)$$

where $\lambda_1 + \lambda_2 + \lambda_3 + \lambda_4 = 1$ and $0 < \lambda_1 < \lambda_2 < \lambda_3 \leq \lambda_4$.

Therefore, the reward function r_t is calculated as

$$r_t = \sum_{k=1}^{\ell} \sum_{f=1}^{\delta_k} r_{k,t}^f \quad (20)$$

where ℓ is the total number of vehicles within the range of the RSU, and δ_k is the number of requested content from V_k .

2) *DRL Algorithm*: DRL has become one of the hotspots in artificial intelligence research. It has been widely used in end-to-end control, recommendation system and other fields. In the vehicle edge network, nodes can interact with the environment and make intelligent decisions [36]. Therefore, here we use DDQN [37] to eliminate the overestimation problem caused by ordinary DQN by decoupling the two steps of target Q value action selection and target Q value calculation.

As mentioned above, when $a_t = 1$, the next state changes. Take the contents of all vehicle requests in the local RSU as input. The target Q value is updated as follows:

$$Q_{\text{target}} = \mathbb{E} \left[r_{t+1} + \gamma Q' \left(s_{t+1}, \arg \max_{a_{t+1}} Q(s_{t+1}, a; \theta_t); \theta'_t \right) \mid (s_t, a_t) \right] \quad (21)$$

where $\gamma \in [0, 1]$ is the discount coefficient used to evaluate the importance of future rewards.

Minimizing the loss function $L(\theta_t)$ is calculated as follows:

$$L(\theta_t) = \mathbb{E} \left[(Q_{\text{target}} - Q(s_t, a_t; \theta_t))^2 \right]. \quad (22)$$

We optimize the loss function by the Adam gradient descent method as follows:

$$\nabla_{\theta_t} L(\theta_t) = \mathbb{E} \left[(Q_{\text{target}} - Q(s_t, a_t; \theta_t)) \nabla_{\theta_t} Q(s_t, a_t; \theta_t) \right]. \quad (23)$$

Through the following formula, the parameter θ'_t of the target Q network can be regularly updated from the latest parameter θ_t of the current Q network to:

$$\theta'_t = \eta \theta'_{t-1} + (1 - \eta) \theta_t \quad (24)$$

where η is the learning rate. At this point, a round of iterations is completed. Because the current slot is t , it is necessary to repeat T iterations before the end of an episode. We will get N popular content, which will be allocated to different cache nodes. The algorithm cycles continuously for a total of E episodes. Finally, the cache nodes update the cache content according to the optimal cooperative cache strategy, and continue algorithm training if the smart vehicles have completed updating the content.

The main calculation process of DDQN is shown in Algorithm 1.

V. PERFORMANCE EVALUATION

A. Simulation Settings

To evaluate the CMCF algorithm proposed in this article, we simulated the urban transportation network environment, as shown in Fig. 1, using Python 3.8 as the simulation tool. This environment consists of cloud servers, RSUs, and intelligent vehicles. The simulation uses the real data set MovieLens1M, which was created by the GroupLens project team of the School of Computer Science and Engineering of the University of Minnesota as a noncommercial research experiment site. MovieLens1M includes 100 0209 ratings for approximately 3800 movies from 6040 anonymous mobile users, divided into different genders, seven age groups, and 20 professions, with ratings ranging from one star to five stars for different release times and categories. The local data used in the simulation of the vehicle comes from this data set. In the data, the training

Algorithm 1 Cooperative Caching Based on DDQN Algorithm

Initialization:

Replay buffer D;

Parameters of the network θ_t ;

Input: The content requested by vehicles of all cluster content under RSU in round r at slot $t \in \{1, 2, 3, \dots, T\}$

for episode $e \in \{1, 2, 3, \dots, E\}$ **do**;

Edge nodes randomly cache c popular content;

Observe the state s_t ;

Get Q_{target} from Eq. (21);

Get state s_{t+1} through action a_t ;

Get reward r_t based on Eq. (20);

Store tuple (s_t, a_t, r_t, s_{t+1}) in D;

Obtain the minimized loss function $L(\theta_t)$ through Formula (22);

Get the gradient $\nabla_{\theta_t} L(\theta_t)$ of the loss function is obtained by Formula (23);

Update the latest parameter θ'_t by Formula (24);

if the slot is T **then**

Update $\theta' = \theta$

TABLE II
VALUES OF THE PARAMETERS IN THE EXPERIMENTS

Notation	Description	Value
B	channel bandwidth	540KHZ
σ^2	noise power	-114dBm
η	learning rate	0.05
$U_{\max}/U_{\min}$	vehicle speed	50km/h—60km/h
P_V	Vehicle Transmission Rate	3 dBm
P_B	RSU Transmission Rate	30 dBm
P_C	Cloud transmit power	43 dBm
γ	discount coefficient	0.9
J	replay buffer	2000
J_e	minibatch	100

set data accounts for 99%, and the test set data accounts for 1%. The request content of the vehicle in the simulation comes from the random data in the data set, and our experimental results are set to the average of the five-round experiment. All vehicles have data for local model training. The default parameter settings of the experiment are shown in Table II.

B. Comparison of Algorithms and Indicators

In order to evaluate the proposed CMCF scheme, we use other caching schemes to compare, as follows.

- 1) *Random Caching (RC)*: Regardless of other factors, RSU and vehicles randomly cache everything.
- 2) *ε -Degree*: Set the cache probability of the content with high popularity in the edge node to $1-\varepsilon$, and the random cache probability to ε . Generally speaking, ε is a small number, so let $\varepsilon = 0.1$.
- 3) *Thompson Sampling (TS)*: First, cache the most popular content in the edge nodes, and then dynamically adjust the proportion of content in the cache according to the probability of previous content requests.

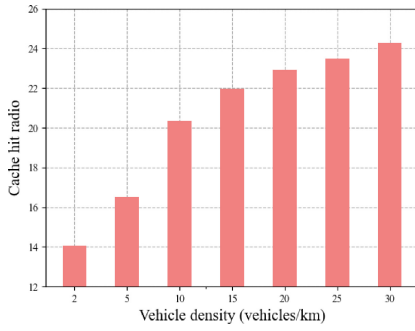


Fig. 6. Effect of vehicle density on cache hit rate.

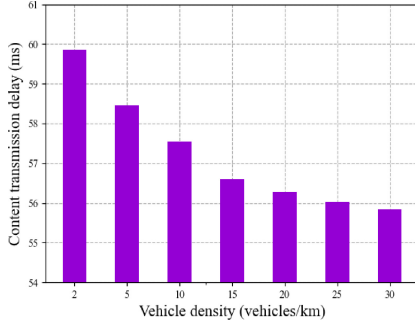


Fig. 7. Effect of vehicle density on content transmission delay.

- 4) *CMCF Without DRL*: After getting popular content, it does not choose the optimal caching strategy through DRL, and caches popular content directly and randomly.

C. Result Analysis and Performance Comparison

Figs. 6 and 7 show the effect of vehicle density on CMCF cache hit ratio and content transmission delay when the total cache capacity of edge cache nodes is 100. From the above picture, it is not difficult to find that the cache hit rate becomes higher and higher with the increase of vehicle density, and the content transmission delay becomes smaller and smaller with the increase of vehicle density. The reason is that when more vehicles enter the coverage area of RSU, the global model trained by RSU will be more accurate, so that popular content can be accurately predicted, and more vehicles can obtain content directly from local. (except for Figs. 6 and 7, the default vehicle density of the experiment is 15 vehicles /km).

Fig. 8 shows the cache hit ratio for different total cache capacity, and all scenarios increase with the increase of the total cache capacity. This is because the larger cache capacity can cache more popular content, and the cache node can better meet the requirements of users. From the figure, we can see that the CMCF strategy proposed in this article is obviously better than other strategies, because the CMCF strategy can better select cache nodes and cache content by predicting the approximate trajectory and popular content of vehicles, and update the cache content in time through the DRL algorithm. When the total cache capacity is 400 MB, the hit ratio of the CMCF algorithm is 4%, 7%, and 50% higher than that of TS, ϵ -greedy and RC, respectively.

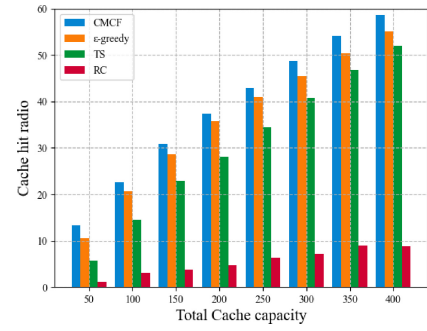


Fig. 8. Effect of total cache capacity on cache hit rate.

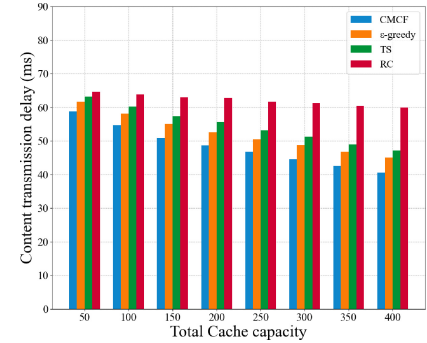


Fig. 9. Effect of total cache capacity on content transmission delay.

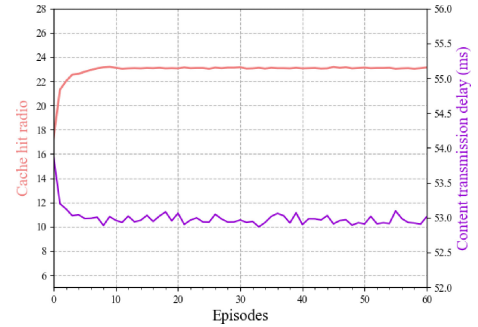


Fig. 10. Changes of cache hit ratio and content transmission delay in DRL with the growth of episode.

Fig. 9 shows the effect of total cache capacity on content delivery latency, and the content transmission delay decreases with the increase of the total cache capacity. This is because a larger cache capacity caches more popular content, making it easier for users to get the requested content. From the figure, we can see that the content transmission delay of the CMCF strategy proposed in this article is significantly lower than that of other strategies, because the CMCF strategy can better select cache nodes and cache content. When the total cache capacity is 400 MB, the content transmission delay of the CMCF algorithm is reduced by 4%, 6%, and 19% compared with TS, ϵ -greedy and RC, respectively.

Fig. 10 shows the cache hit ratio and content delivery latency of the CMCF strategy using DRL in different episodes. When the total cache capacity is 100 and the episodes is 1–5 episodes, the two parameters changed greatly. When the

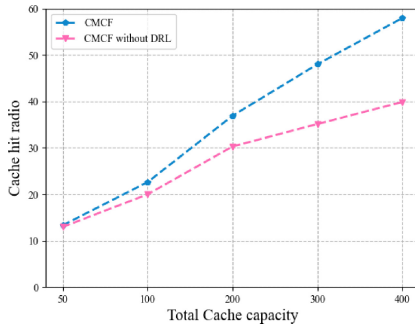


Fig. 11. Effect of cache capacity change on hit rate of CMCF cache with or without DRL.

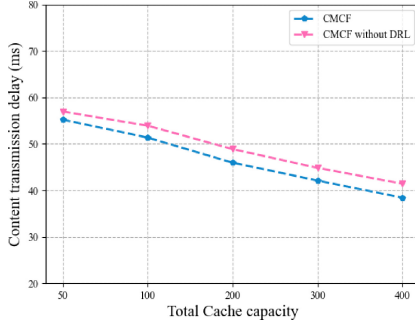


Fig. 12. Effect of the change of cache capacity on the transmission delay of CMCF content with or without DRL.

episodes is 5–10 episodes, the two parameters tend to stabilize. It is shown that DRL converges around 10 episodes and the optimal cooperative caching strategy is obtained.

Figs. 11 and 12 show that the CMCF using DRL can achieve better cache hit rates and lower content transmission latency. First, with the increase of the total cache capacity, the cache hit rate of the CMCF with DRL is better than that of the scheme without DRL, because the optimal cooperative cache strategy can be obtained through DRL to optimize the cache content. Second, with the increase of the total cache capacity, we use DRL's CMCF strategy to get lower content transmission delay, because the optimal cooperative cache strategy optimizes the cache content of the node and makes it easier for users to obtain the requested content.

VI. CONCLUSION

In this article, we propose a collaborative edge caching framework based on location prediction, federated learning, and DRL to solve the problem of high-latency and low-hit rate of content delivery in vehicular edge networks. Different from other caching strategies, we propose a federated learning algorithm combined with AE based on dynamic vehicle clusters, and use consistent hashing to allocate cooperative content. Finally, the optimal cooperative caching strategy is obtained by using DRL. In addition, the superiority of the scheme is proved by experiments.

REFERENCES

[1] L. Liu, C. Chen, Q. Pei, S. Maharjan, and Y. Zhang, "Vehicular edge computing and networking: A survey," *Mobile Netw. Appl.*, vol. 26, no. 3, pp. 1145–1168, 2021.

[2] L. Zhao, Y. Ran, H. Wang, J. Wang, and J. Luo, "Towards cooperative caching for vehicular networks with multi-level federated reinforcement learning," in *Proc. IEEE Int. Conf. Commun.*, 2021, pp. 1–6.

[3] Z. Zhang, C.-H. Lung, M. St-Hilaire, and I. Lambadaris, "Smart proactive caching: Empower the video delivery for autonomous vehicles in ICN-based networks," *IEEE Trans. Veh. Technol.*, vol. 69, no. 7, pp. 7955–7965, Jul. 2020.

[4] T. X. Tran and D. Pompili, "Adaptive bitrate video caching and processing in mobile-edge computing networks," *IEEE Trans. Mobile Comput.*, vol. 18, no. 9, pp. 1965–1978, Sep. 2019.

[5] H. Wu, Y. Fan, Y. Wang, H. Ma, and L. Xing, "A comprehensive review on edge caching from the perspective of total process: Placement, policy and delivery," *Sensors*, vol. 21, no. 15, p. 5033, 2021.

[6] Y. Zhang, R. Wang, M. S. Hossain, M. F. Alhamid, and M. Guizani, "Heterogeneous information network-based content caching in the Internet of Vehicles," *IEEE Trans. Veh. Technol.*, vol. 68, no. 10, pp. 10216–10226, Oct. 2019.

[7] S. Xu, X. Liu, S. Guo, X. Qiu, and L. Meng, "MECC: A mobile edge collaborative caching framework empowered by deep reinforcement learning," *IEEE Netw.*, vol. 35, no. 4, pp. 176–183, Jul./Aug. 2021.

[8] Y. Zhang, C. Li, T. H. Luan, Y. Fu, W. Shi, and L. Zhu, "A mobility-aware vehicular caching scheme in content centric networks: Model and optimization," *IEEE Trans. Veh. Technol.*, vol. 68, no. 4, pp. 3100–3112, Apr. 2019.

[9] B. Ko, K. Liu, S. H. Son, and K.-J. Park, "RSU-assisted adaptive scheduling for vehicle-to-vehicle data sharing in bidirectional road scenarios," *IEEE Trans. Intell. Transp. Syst.*, vol. 22, no. 2, pp. 977–989, Feb. 2021.

[10] Q. Wu, Y. Zhao, Q. Fan, P. Fan, J. Wang, and C. Zhang, "Mobility-aware cooperative caching in vehicular edge computing based on asynchronous federated and deep reinforcement learning," *IEEE J. Sel. Topics Signal Process.*, vol. 17, no. 1, pp. 66–81, Jan. 2023.

[11] L. Yao, Y. Wang, X. Wang, and G. Wu, "Cooperative caching in vehicular content centric network based on social attributes and mobility," *IEEE Trans. Mobile Comput.*, vol. 20, no. 2, pp. 391–402, Feb. 2021.

[12] W. Huang, T. Song, Y. Yang, and Y. Zhang, "Cluster-based cooperative caching with mobility prediction in vehicular named data networking," *IEEE Access*, vol. 7, pp. 23442–23458, 2019.

[13] G. Deng, F. Li, and L. Wang, "Cooperative downloading in VANETs-LTE heterogeneous network based on named data," in *Proc. IEEE Conf. Comput. Commun. Workshops (INFOCOM WKSHPS)*, 2016, pp. 233–238.

[14] R. Wang, Z. Kan, Y. Cui, D. Wu, and Y. Zhen, "Cooperative caching strategy with content request prediction in Internet of Vehicles," *IEEE Internet Things J.*, vol. 8, no. 11, pp. 8964–8975, Jun. 2021.

[15] X. Zhang, Z. Qi, G. Min, W. Miao, Q. Fan, and Z. Ma, "Cooperative edge caching based on temporal convolutional networks," *IEEE Trans. Parallel Distrib. Syst.*, vol. 33, no. 9, pp. 2093–2105, Sep. 2022.

[16] X. Huang, K. Xu, Q. Chen, and J. Zhang, "Delay-aware caching in Internet-of-Vehicles networks," *IEEE Internet Things J.*, vol. 8, no. 13, pp. 10911–10921, Jul. 2021.

[17] J. Zhao, X. Sun, Q. Li, and X. Ma, "Edge caching and computation management for real-time Internet of Vehicles: An online and distributed approach," *IEEE Trans. Intell. Transp. Syst.*, vol. 22, no. 4, pp. 2183–2197, Apr. 2021.

[18] D. Gupta, S. Rani, S. H. Ahmed, S. Garg, M. J. Piran, and M. Alrashoud, "ICN-based enhanced cooperative caching for multimedia streaming in resource constrained vehicular environment," *IEEE Trans. Intell. Transp. Syst.*, vol. 22, no. 7, pp. 4588–4600, Jul. 2021.

[19] L. Yao, A. Chen, J. Deng, J. Wang, and G. Wu, "A cooperative caching scheme based on mobility prediction in vehicular content centric networks," *IEEE Trans. Veh. Technol.*, vol. 67, no. 6, pp. 5435–5444, Jun. 2018.

[20] G. Qiao, S. Leng, S. Maharjan, Y. Zhang, and N. Ansari, "Deep reinforcement learning for cooperative content caching in vehicular edge computing and networks," *IEEE Internet Things J.*, vol. 7, no. 1, pp. 247–257, Jan. 2020.

[21] K. Zhang, J. Cao, S. Maharjan, and Y. Zhang, "Digital twin empowered content caching in social-aware vehicular edge networks," *IEEE Trans. Comput. Social Syst.*, vol. 9, no. 1, pp. 239–251, Feb. 2022.

[22] M. K. Denko and J. Tian, "Cross-layer design for cooperative caching in mobile ad hoc networks," in *Proc. 5th IEEE Consum. Commun. Netw. Conf.*, 2008, pp. 375–380.

[23] X. Wang, C. Wang, X. Li, V. C. M. Leung, and T. Taleb, "Federated deep reinforcement learning for Internet of Things with decentralized cooperative edge caching," *IEEE Internet Things J.*, vol. 7, no. 10, pp. 9441–9455, Oct. 2020.

- [24] A. Ndikumana, N. H. Tran, D. H. Kim, K. T. Kim, and C. S. Hong, "Deep learning based caching for self-driving cars in multi-access edge computing," *IEEE Trans. Intell. Transp. Syst.*, vol. 22, no. 5, pp. 2862–2877, May 2021.
- [25] D. T. Hoang, D. Niyato, D. N. Nguyen, E. Dutkiewicz, P. Wang, and Z. Han, "A dynamic edge caching framework for mobile 5G networks," *IEEE Wireless Commun.*, vol. 25, no. 5, pp. 95–103, Oct. 2018.
- [26] Z. Yu, J. Hu, G. Min, Z. Zhao, W. Miao, and M. S. Hossain, "Mobility-aware proactive edge caching for connected vehicles using federated learning," *IEEE Trans. Intell. Transp. Syst.*, vol. 22, no. 8, pp. 5341–5351, Aug. 2021.
- [27] T. Ma, X. Chen, Z. Ma, and Y. Chen, "Deep reinforcement learning for pre-caching and task allocation in Internet of Vehicles," in *Proc. IEEE Int. Conf. Smart Internet Things (SmartIoT)*, 2020, pp. 79–85.
- [28] "Study on LTE-based V2X services," 3rd Gener. Partnership Project (3GPP), Sophia Antipolis, France, Rep. TR 36.885, 2016.
- [29] J. Chen, H. Wu, P. Yang, F. Lyu, and X. Shen, "Cooperative edge caching with location-based and popular contents for vehicular networks," *IEEE Trans. Veh. Technol.*, vol. 69, no. 9, pp. 10291–10305, Sep. 2020.
- [30] J. Silva, N. Varela, O. B. P. Lezama, V. Álvarez, and B. D. L. Hoz, "Vehicle flow prediction through probabilistic modeling," in *Proc. Int. Conf. Res. Appl. Defense Secur.*, 2020, pp. 409–417.
- [31] F. N. Neto, C. Baptista, and C. Campelo, "Predicting routes and destinations of urban trips using PPM method," in *Proc. SBC*, 2015, pp. 121–130.
- [32] W. Xie and Y. Chen, "Elastic consistent hashing for distributed storage systems," in *Proc. IEEE Int. Parallel Distrib. Process. Symp. (IPDPS)*, 2017, pp. 876–885.
- [33] V. Mirrokni, M. Thorup, and M. Zadimoghaddam, "Consistent hashing with bounded loads," in *Proc. 29th Annu. ACM-SIAM Symp. Discr. Algorithms*, 2018, pp. 587–604.
- [34] J. Konečný, H. B. McMahan, F. X. Yu, P. Richtárik, A. T. Suresh, and D. Bacon, "Federated learning: Strategies for improving communication efficiency," 2016, *arXiv:1610.05492*.
- [35] A. Ng et al., "Sparse autoencoder," Lecture Notes CS294A, Stanford Univ., Stanford, CA, USA, 2011.
- [36] N. Lyu, G. Song, B. Yang, and Y. Cheng, "QNGPSR: A Q-network enhanced geographic ad-hoc routing protocol based on GPSR," in *Proc. IEEE 88th Veh. Technol. Conf. (VTC-Fall)*, 2018, pp. 1–6.
- [37] H. Van Hasselt, A. Guez, and D. Silver, "Deep reinforcement learning with double Q-learning," in *Proc. AAAI Conf. Artif. Intell.*, vol. 30, 2016, pp. 2094–2100.



Honghai Wu received the B.S. degree from Zhengzhou University, Zhengzhou, China, in 2001, and the M.S. and Ph.D. degrees from Beijing University of Posts and Telecommunications, Beijing, China, in 2007 and 2015, respectively.

He worked with China United Telecommunications Co. Ltd., Beijing, China, from 2007 to 2011. He is currently working with the College of Information Engineering, Henan University of Science and Technology, Luoyang, China. His research interests include mobile edge

computing, smart car network, and video data delivery.



Jichong Jin received the B.S. degree in software engineering from Huanghuai University, Zhumadian, China, in 2020. He is currently pursuing the M.S. degree in computer science and technology with Henan University of Science and Technology, Luoyang, China.

His research interests is edge caching.



Huahong Ma received the B.S. degree from Zhengzhou University, Zhengzhou, China, in 2001, the M.S. degree from Yunnan University, Kunming, China, in 2005, and the Ph.D. degree from Henan University of Science and Technology, Luoyang, China, in 2019.

She worked with the College of Information Engineering, Henan University of Science and Technology since 2005. She is currently an Associate Professor and her main research interests are crowd sensing network and Internet of Things.



Ling Xing received the B.S. degree in electronic engineering from the Southwest University of Science and Technology, Mianyang, China, in 2002, the M.S. degree from the University of Science and Technology of China, Hefei, China, in 2005, and the Ph.D. degree in communications and information systems from Beijing Institute of Technology, Beijing, China, in 2008.

She is currently a Professor with the School of Information Engineering, Henan University of Science and Technology, Luoyang, China. Her current research interests include intelligent information process, information semantic analysis, and multimedia computing and networking.